– Independent Work Report Spring 2026 –

Time Series Forecasting with Universal Sequence Preconditioning*

Alex Kawaja
Advisor: Elad Hazan

Abstract

Universal Sequence Preconditioning (USP) is a recent theoretical framework that applies a fixed convolutional transformation to the target sequence, leading to provable guarantees for online prediction algorithms on data arising from marginally stable linear dynamical systems. Meanwhile, zero-shot time series forecasting models such as Moirai-2 have achieved strong results across a diverse set of domains. This paper asks whether USP can improve the performance of these zero-shot models. We find that the most direct application of USP, which involves training the model to predict the preconditioned target and reversing at inference, fails in practice because the reversal causes prediction error to compound step-by-step and degrades performance, despite the preconditioned time series being measurably easier to learn. We propose polynomial hint preconditioning*, which instead uses the preconditioned signal as an auxiliary input channel to the model while keeping the original target. On GIFT-Eval (97 configurations, 5 seeds), our hint preconditioning leads to a 2.90% geometric mean MASE improvement over the baseline Moirai-2 Small model ($p = 9.6 \times 10^{-7}$) with only 0.11% more parameters. Capacity controls confirm that the improvement is caused by the polynomial content of the hint channel, not by additional parameters.*

1. Introduction

Accurate zero-shot time series forecasting can be useful in many domains including energy, weather, economics, and healthcare. They can be especially valuable when reliable predictions are needed but training data is scarce. Recent *universal* time series foundation models including Moirai [7, 5],

*Code available at `NeedtoaddURL`.

TimesFM [4], and Chronos [2] train a single large transformer model on diverse datasets, which can then generalize zero-shot to unseen data. These models learn general patterns and can often compete with or outperform models trained specifically for one domain.

Separately, Marsden and Hazan [6] have shown that by applying *Universal Sequence Preconditioning* (USP), a fixed convolutional transformation to the target sequence, prediction methods like spectral filtering and convex regression can learn marginally stable systems with sublinear regret. The transformation is universal in that it doesn't require learnable parameters and only depends on the choice of convolutional coefficients. This raises our central question:

> *Can the theoretical benefits of universal sequence preconditioning be realized in modern time series foundation models?*

The goal of this project is to answer this question empirically for the Moirai-2 model.

The most direct application of USP to a foundation-model setting is to train the model to predict the preconditioned target sequence and then reverse the transformation on the model's predictions to recover the original signal. We find that the preconditioned target is indeed easier to learn. The model trained on preconditioned data has lower loss throughout training, and when we reverse the predictions with ground truth values (oracle reversal), it improves performance significantly. But as we will see, reversing in a causal way must use the model's own predictions, and these errors compound step by step, leading to performance worse than the baseline model.

We therefore propose a second approach: *polynomial hint preconditioning*. Instead of transforming the target, we add the preconditioned signal as an auxiliary input channel to the model along with the original series. The model still predicts in the original space, so no reversal is needed. This seems to allow the model to utilize some of the benefit that the preconditioning provides while avoiding the error-accumulation caused by the preconditioning reversal.

On the GIFT-Eval benchmark [1], our hint preconditioning method improves Moirai-2 Small's geometric mean MASE by 2.90% ($p = 9.6 \times 10^{-7}$). Our expanded input channel only adds 12,288 parameters (0.11% of the model), and requires no change to the transformer architecture. Experiments where we controlled for this parameter increase by replacing the polynomial hint with

either zeros or a duplicate of the input channel perform worse than our hint model, confirming that the improvement can be attributed to the preconditioned content and not the extra parameters.

Contributions. This paper makes the following contributions:

1. We identify and quantify the issues with directly applying USP to autoregressive forecasters by training on the preconditioned sequence. Specifically, the causal reversal process compounds error and negates the theoretical benefit.
2. We introduce hint preconditioning as an alternative application of USP that avoids this error accumulation by providing the preconditioned signal as an auxiliary input.
3. We tested varying the choice of polynomial degree, stride, and hint dropout and identified that degree d=4 Chebyshev with stride equal to patch size and 10% dropout resulted in the best performance.

2. Background and Related Work

2.1. Classical Sequence Preconditioning

Preconditioning is a technique that involves applying a transformation to the input sequence with the goal of making the resulting sequence easier to predict. One example of this is *differencing*, introduced by Box and Jenkins [3]. Differencing transforms a sequence using the transformation $\tilde{y}_t = y_t - y_{t-1}$. This can be helpful because many time series are not stationary and differencing can help eliminate linear trends.

One way to think about the differencing operation is a special case of a convolution over previous time steps, namely $\tilde{y}_t = \sum_{i=0}^{n} c_i y_{t-i}$. The question, then, is whether there is a way to effectively choose those coefficients c_i in a way that further improves the predictability of the preconditioned sequence. Universal Sequence Preconditioning, which we describe next, can help answer that question.

2.2. Universal Sequence Preconditioning

Marsden and Hazan [6] are able to show mathematically the effect a fixed convolutional transformation like this has on a sequence, by assuming that the sequence was generated by a marginally stable linear dynamical system. They use this to find a set of fixed coefficients that leads to sublinear regret bounds for standard online prediction algorithms. This transformation with these fixed coefficients is known as Universal Sequence Preconditioning (USP).

To build intuition, consider a linear dynamical system with state transition matrix A, input matrix B, and observation matrix C:

$$x_{t+1} = Ax_t + Bu_t, \qquad y_t = Cx_t. \tag{1}$$

As we saw above, given coefficients $1, c_1, \ldots, c_n$, define the *preconditioned target*

$$\tilde{y}_t = y_t + \sum_{i=1}^{n} c_i y_{t-i}. \tag{2}$$

By substituting the definition of y_t from the LDS definition and simplifying algebraically, Marsden and Hazan show that $\tilde{y}_t$ can be decomposed into two terms:

$$\tilde{y}_t = \aleph_1(A,B,C) + Cp_n^c(A)\aleph_2(A,B) \tag{3}$$

where

$$p_n^c(x) = \sum_{i=0}^{n} c_i x^{n-i} \tag{4}$$

The first term $\aleph_1$ is a finite sum that can be learned by convex regression. The second term $\aleph_2$ can be minimized by the correct choice of coefficients c_i, particularly by minimizing the operator norm $\|p_n^c(A)\|$. In fact, if we knew what A was it would be possible to completely eliminate this term with the Cayley-Hamilton theorem, but instead we are looking for a choice of c_i that minimizes this norm in a more general case.

Why Chebyshev As we saw, the goal is to choose coefficients that minimize $\|p_n^c(A)\|$ for a more general A. For marginally stable A (A has eigenvalues in $[-1,1]$) a monic Chebyshev polynomial works well. This is because

$$\max_{\lambda \in [-1,1]} |p_n(\lambda)| \leq 2^{-(n-1)}. \tag{5}$$

As a result, if A is diagonalizable, preconditioning using coefficients from a monic chebyshev polynomial will shrink $\aleph_2$ exponentially, and allow for low regret on the preconditioned sequence.

Our work asks whether these theoretical benefits in online learning can transfer to our setting of zero-shot time series forecasting. These models are different from the original setting in that they are trained offline, they are nonlinear, they operate on patches instead of individual timesteps, and the "data" they see during pretraining is a mixture of many different time series with different underlying systems.

2.3. Time Series Foundation Models

Time series foundation models are trained on large collections of different time series and aim for strong performance on benchmarks in a zero-shot setting. Recently, the Transformer architecture has become the dominant approach for this task, and many of the current high-performing models transform the time series into a sequence of embedding vectors that they pass through a transformer. For example, Amazon's Chronos models [2] tokenize continuous values into discrete bins. Google's TimesFM [4] and the Salesforce Moirai family [7, 5] separate the input sequence into blocks, or patches, which are then transformed individually into embedding vectors.

Our experiments build on Moirai-2 Small [5]. Moirai-2 Small is an 11.4M parameter decoder only transformer. It normalizes the input sequence and breaks it into non-overlapping patches of constant size $P = 16$. Each patch is transformed into an embedding vector through a residual block, and the embeddings are then passed into a transformer. Finally, an output head predicts 9 quantile levels for each time step that are optimized with quantile loss.

Because each patch embedding initially encodes blocks of 16 values, information that spans multiple patches must be recovered by self-attention in the transformer. Another advantage of

our preconditioning technique is that it allows some cross-patch information to be included in the embeddings themselves.

2.4. Benchmark: GIFT-Eval

GIFT-Eval [1] is a standard zero-shot time series forecasting benchmark. It contains 24 datasets with over 144k time series and 177 million data points. We use all 97 evaluation configurations that span multiple domains, frequencies, and prediction lengths. All quantitative results in this paper are reported on GIFT-Eval.

3. Approach

3.1. Notation

Moirai-2 takes as input a univariate time series $x = (x_1, \ldots, x_T)$ and an observation mask m. After normalization, x is partitioned into $\lceil T/P \rceil$ patches $p_i = (x_{(i-1)P+1}, \ldots, x_{iP})$ with $P = 16$. Each patch and its corresponding mask are concatenated and transformed to an embedding through a residual block:

$$e_i = \text{ResBlock}_{2P \to d}([p_i; m_i]). \tag{6}$$

A transformer encoder processes $(e_1, \ldots, e_N)$ and a quantile head produces multi-step forecasts.

3.2. Approach #1: Direct Preconditioning with Reversal

The most direct way to apply USP to the time series forecasting domain is to train the model on the preconditioned target $\tilde{y}_t = \sum_i c_i y_{t-i}$ and invert at inference to recover the prediction in the original space. In this case, the only difference in training is that the model's loss is computed against the preconditioned values $\tilde{y}$ rather than y. During inference, we reverse the models predictions with the formula:

$$\hat{y}_t = \hat{\tilde{y}}_t - \sum_{i=1}^{n} c_i \hat{y}_{t-i}. \tag{7}$$

We call this reversal method *causal reversal* because it uses only the model's previous predictions to reverse the preconditioning. In contrast, the *oracle reversal* model substitutes ground-truth y_{t-i} into

(7), which is useful for obtaining an upper bound on how well the model can learn preconditioned data. Causal reversal is necessary outside of a testing environment because future ground truth values are not available.

The motivation for this approach is clear, as USP theory predicts that the preconditioned signal is easier to learn. Later we will see that we can actually confirm this empirically (Section 5.1). However, the issue is that (7) is autoregressive and causes errors to compound. For example, an error in $\hat{y}_{t-1}$ propagates into the formula when computing $\hat{y}_t$, which then affects $\hat{y}_{t+1}$, and so on. In Section 5.1 we show that this issue of error accumulation is worse than the gains we see from learning in the preconditioned space and leaves the method worse than the baseline.

3.3. Approach #2: Polynomial Hint Preconditioning

In order to combat this problem, we designed a method which made predictions in the original space, leaving the target unchanged, in order to avoid the compounding errors. Instead, we provided the USP preconditioned signal as an auxiliary input to the model, allowing the model to utilize information that it provides about the input dynamics.

More specifically, we add what we call a *hint channel* by applying the convolution filter with Chebyshev coefficients to the normalized input series before patching. For polynomial degree d and stride s, the hint at time t is

$$h_t = \sum_{i=0}^{n} c_i x_{t-is}, \qquad \tilde{h}_t = h_t - x_t, \tag{8}$$

where $c_0, \ldots, c_n$ are the coefficients of the monic Chebyshev polynomial of degree n. We find that in this scenario, subtracting the self-term so that the hint provides only new information when concatenated to the patch yields stronger performance. We also introduce a stride parameter to the preconditioning which is optimally set at the patch size $P = 16$ such that the filter is applied to corresponding positions in each of the patches. This can also be thought of as applying the preconditioning transformation to the patch vectors themselves. We confirm this empirically in Section 5.3.

The hint is partitioned into patches aligned with the input patches, and each patch embedding

now takes a $3P$-dimensional input:

$$e_i = \text{ResBlock}_{3P \to d}([p_i; m_i; \tilde{h}_i]). \tag{9}$$

Only the first linear layer of the ResBlock widens (from $2P$ to $3P$ inputs). The transformer encoder and output head are unchanged. Compared to the baseline embedding transformation, this adds 12,288 parameters out of 11.4M total, which amounts to a 0.11% increase.

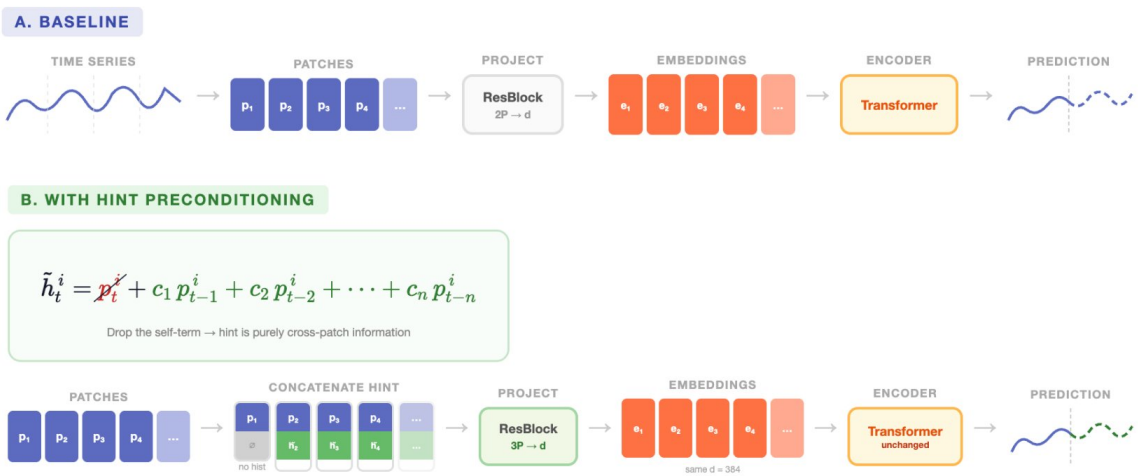

Figure 1: The baseline Moirai-2 pipeline (top) versus the hint preconditioning pipeline (bottom). Only the input projection widens (from $2P \to d$ to $3P \to d$); the transformer encoder is unchanged.

Hint dropout We found that we could improve performance through what we call hint dropout. With some probability p_{drop}, each patch's hint is independently set to zero. This helps prevent the model from relying too much on the hint channel. At inference, the full hint is always provided. We find $p_{\text{drop}} = 0.1$ to be optimal (Section 5.3). This led to the naming convention HD10 for our model.

4. Implementation

4.1. Overview

All of our preconditioning experiments are built on top of the Salesforce `uni2ts` library [7]. This library includes full training code for the Moirai-1 model as well as the PyTorch modules for Moirai-2. Our modifications span a few areas, including the input projection (for hint channel),

the data loader (for outlier filtering), and inference logic (for preconditioning reversal). Sections 4.2–4.7 describe the implementation choices we made on top of the `uni2ts` library in more detail.

4.2. Base Model and Training Data

Moirai-2 Small We chose the Moirai family of models because of the models currently on the GIFT-eval leaderboard, Moirai was the highest performing model that released training code. We chose Moirai-2 Small model instead of Base or Large because the Moirai-2 paper shows that the Small model performs best on GIFT-Eval. It was also more efficient to run experiments on the Small model given limited compute.

Training data We train on LOTSA [7], a publicly available collection of time series containing 27B observations that was released by Salesforce with Moirai 1.0. The official Moirai-2 model was trained with a significant amount of additional data that was not publicly available including synthetic data and Salesforce internal data (295B total observations). For this reason, our baseline model, which is the Moirai-2 Small architecture trained on LOTSA data, results in a higher MASE on GIFT-Eval than the official Moirai-2 Small model. All of the models in our experiments are trained identically on LOTSA, using variate-proportional sampling for multivariate series.

4.3. Preconditioning

Approach #1: Reversal For Approach #1, where the model learns in the preconditioned space, we apply USP preconditioning to the input series before it enters the model. More specifically, the input sequence is first normalized, then preconditioned, and then fed to the model. The model sees preconditioned values as context and learns to predict in the preconditioned space with a quantile loss against $\tilde{y}$. At inference, the model produces a preconditioned quantile prediction $\hat{\tilde{y}}$. In the causal model, we apply the inverse transformation $\hat{y}_t = \hat{\tilde{y}}_t - \sum_{i=1}^{n} c_i \hat{y}_{t-i}$ to each quantile of the prediction for each time step, but always using the median quantile from $\hat{y}_{t-i}$. This occurs unless $t-i$ is in the context, in which case we use the ground truth y_{t-i}.

Approach #2: Hint Channel The hint signal h_t is calculated with a 1D convolution over the normalized time series, with the fixed coefficients of the monic Chebyshev polynomial and stride 16 equal to the patch size of Moirai-2. Then, the original normalized signal is subtracted to produce $\tilde{h}_t$. Finally, the input and hint signals are patched and sent to the residual block. In the residual block, which transforms the patches into embeddings, both the hidden and skip linear layers have their input dimension extended from $2P$ to $3P$. Also, for the first 64 positions of context, or in general degree $\times$ stride positions where there isn't enough history to compute the full convolution, we use a mask to set the hint to zero. These points still contribute normally to the loss.

4.4. Training Setup

Our training hyperparameters are the same as those published for the official Moirai-2 model [5], namely: AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay 0.1; cosine learning rate schedule with peak $\eta = 10^{-3}$ and a 1,000-step linear warmup; batch size 256; bf16 mixed precision. Note that the official model has a 10,000 step linear warmup, which we modified proportional to our total step count. During data loading, we apply z-score anomaly filtering to exclude loss from any training samples which contain a point larger than 8 standard deviations away from the mean. The Moirai-2 paper mentioned filtering samples with significant statistical deviation, and we found that a threshold of 8 worked well for stabilizing training but not removing too many samples. We also trained each model using 5 different seeds (0, 1, 2, 7, 42) to minimize the impact of random sampling in the results.

Training length We decided to train our models for 10,000 steps rather than the 100,000 step pipeline used in the official Moirai-2 model. We made this choice because we found that the official model overfit our smaller dataset after 100,000 steps and its performance on GIFT-eval significantly worsened. In fact, the performance gap with preconditioning increased after 100,000 steps. Therefore, we decided to use 10,000 steps as the setting that most faithfully evaluates hint preconditioning against a high performing baseline on the available data.

4.5. Evaluation Protocol

We evaluated the models on all 97 GIFT-Eval [1] configurations with a context length of 4,000, which matches what Salesforce used for Moirai-2 on the GIFT-eval leaderboard. The metric we use is the geometric mean MASE across configurations. We also calculate per-configuration MASE for statistical testing which is described more below.

In the evaluation we report the geometric mean of raw MASE, which is the prediction error divided by the in-sample seasonal naive error for each series. This is slightly different from the scores that appear on the GIFT-eval leaderboard, which are additionally divided by the seasonal naive baseline's aggregate score (normalized MASE). The relative improvements are identical regardless of the metric.

Statistical testing For each of our models, we train 5 versions, each one using a different seed, and record the MASE score for each config of GIFT-eval. Then, we average the MASE per configuration across the 5 seeds, giving 97 averaged scores per model. Finally, when comparing one model to another, we use a two-sided paired sign test on the number of wins, or lower MASE values, out of the 97 tests.

5. Evaluation

5.1. Approach #1: Why Direct Preconditioning Fails

Our approach #1 Model was trained on the preconditioned target $\tilde{y}_t = \sum_{i=0}^{5} c_i y_{t-i}$ with degree-5 Chebyshev coefficients. We evaluated this model with the following two reversal strategies on GIFT-eval.

- Oracle reversal uses ground-truth y_{t-i} in (7) to reverse preconditioning. Its performance measures how well the model can learn preconditioned data, and gives an upper bound for a model that predicts in preconditioned space.
- Causal reversal uses the model's own predictions $\hat{y}_{t-i}$ in (7) to reverse the preconditioning.

Figure 2 shows the result. The oracle reversal improves geometric mean MASE relative to

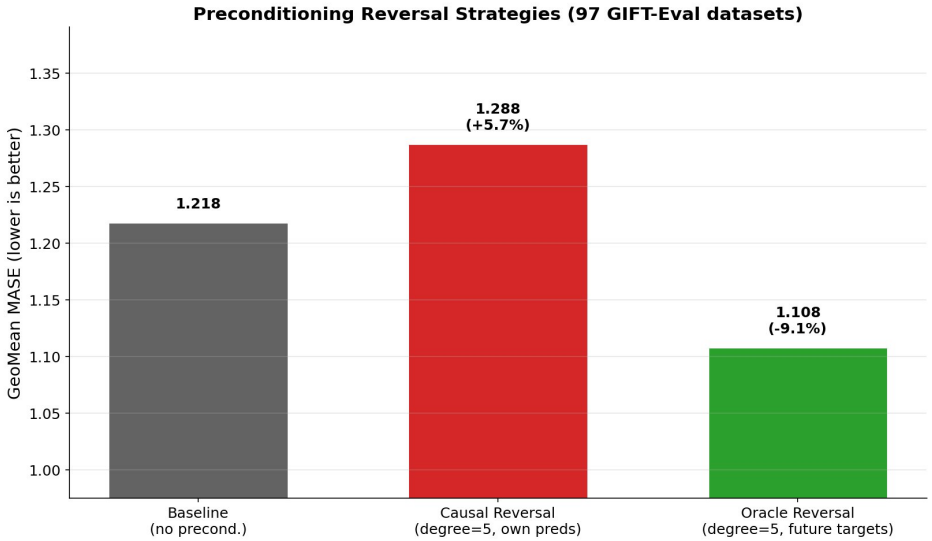

Figure 2: Geometric mean MASE on GIFT-Eval (97 configurations) for direct preconditioning with the two reversal strategies. Oracle reversal improves MASE by 9.1%, showing the preconditioned signal is easier to learn. But causal reversal worsens MASE by 5.7% due to error accumulation.

the baseline by 9.1%. This provides evidence for the idea that we saw in USP theory that the preconditioned signal is easier to learn. This idea is reinforced by Figure 3 which shows the training loss curves for both the baseline model and the one trained on preconditioned targets. We see that the preconditioned model has consistently lower quantile loss throughout training. Then we consider the causal reversal. Note that the underlying model is the same in both cases, so this model predicts preconditioned targets as wel as the oracle model can. However, the error accumulation seen during the reversal led to a geometric mean MASE of 1.288, which was 5.7% worse than baseline, and a 14.8% worse than the oracle model.

Figure 4 is able to highlight error accumulation as a cause for this performance degradation. We see that per-step absolute error for the two reversal strategies is similar for the first few steps, but starts to diverge as the predictions become more and more affected by errors in previous forecasts. By step 30, the causal reversal error is 26% higher than oracle reversal error.

These results, along with others in which we attempted to regularize the preconditioning coefficients to mitigate this issue, led us to the idea that in order to see performance improvements we needed to avoid the preconditioning reversal. This led us to consider Approach #2.

Figure 3: Training loss (quantile loss) comparison over steps 20,000–100,000 between the baseline and a model trained to predict the preconditioned target. The preconditioned model shows lower loss throughout training.

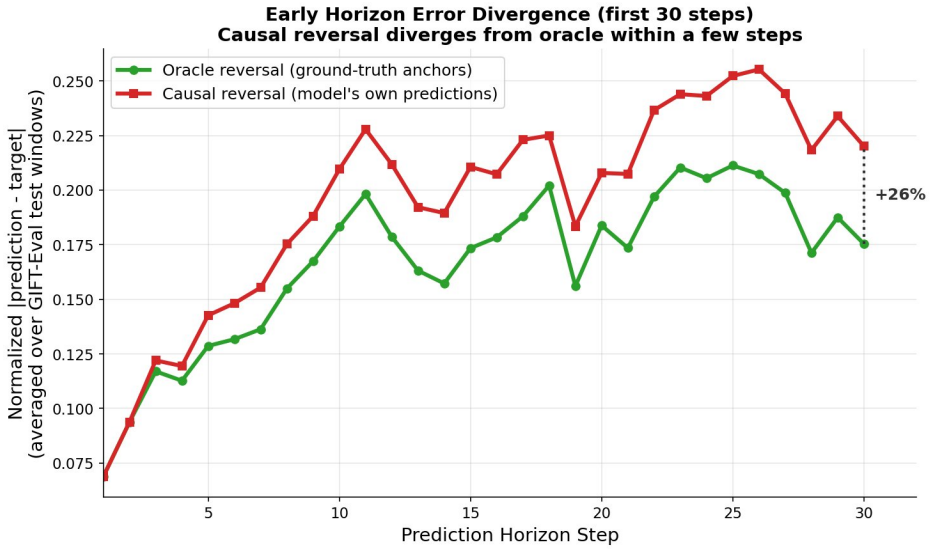

Figure 4: We averaged the mean absolute prediction error by forecast step across 202 test windows samples from 8 GIFT-eval configurations. We find that the oracle and causal start with similar errors, but diverge within a few steps.

5.2. Main Result: Hint Preconditioning vs. Baseline and Capacity Controls

Table 1 and Figure 5 show the main findings. They compare our HD10 model, which uses a Chebyshev degree 4 hint with 10% dropout, to Baseline and the two parameter matched controls.

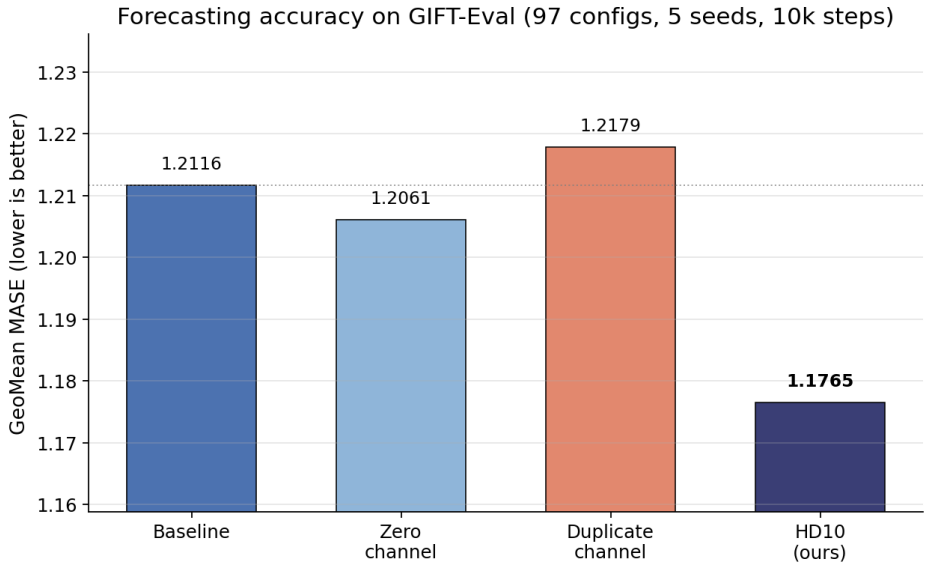

Figure 5: Geometric mean MASE averaged across 5 seeds on GIFT-Eval. Our HD10 model significantly outperforms the baseline and both parameter-matched controls.

Comparison	$\Delta\%$ MASE	Wins/97	p-value
HD10 vs. Baseline	-2.90%	72	9.6×10^{-7}
HD10 vs. Zero channel	-2.45%	77	8.3×10^{-9}
HD10 vs. Duplicate channel	-3.39%	77	8.3×10^{-9}

Table 1: Pairwise model comparisons MASE (geometric mean across seeds, results in 97 observations per comparison). p-values are from a two-sided paired sign test.

HD10 achieves a geometric mean MASE of 1.1765, a 2.90% improvement over the baseline (1.2116), winning 72 of 97 comparisons ($p = 9.6 \times 10^{-7}$). In order to confirm that the performance increase was due to the preconditioning hint content instead of the 0.11% increase in parameters, we also compared our HD10 model against controls that filled the hint with either zeros or a duplicate of the input. Our model had an even higher win rate against these models, indicating that the performance improvement can be attributed to the preconditioned content of the additional input channel.

Figure 6 shows that the preconditioning hint also reduces the model's quantile loss throughout further training, indicating that additional training of the model does not cause the baseline to converge to our HD10 model.

Figure 6: Training loss (quantile loss) for HD10 and the baseline over steps 20,000–100,000. HD10 maintains a lower loss throughout training.

5.3. Ablation Studies

We perform ablations by testing learned filter coefficients and adjusting polynomial degree, stride length, and hint dropout rate. All of the ablations use seed 0 and are otherwise identical to the HD10 model.

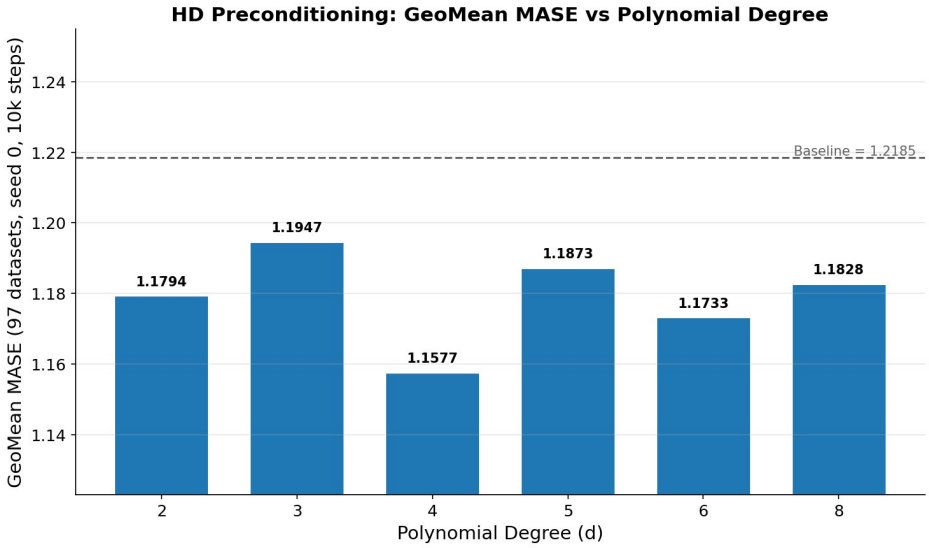

Figure 7: Polynomial degree ablation with seed 0 on all 97 configurations.

Learned Coefficients Instead of fixing the coefficients used to generate the hint as the coefficients of the degree 4 monic Chebyshev polynomial, we tried training the model with those coefficients as learnable parameters. In one experiment, these learnable parameters were initialized to be the Chebyshev polynomial, and in another they were initialized to be zero. On the GIFT-eval for the models trained with seed 0, the Chebyshev initialized and zero initialized models scored 1.172 and 1.175 respectively, underperforming our HD10 model's score of 1.158. We also tested the zero initialized learned coefficient model across all 5 seeds, and found that our HD10 model with fixed coefficients has a 0.3% lower MASE, and won 59% of individual GIFT-eval configurations. It is very interesting to see that in this case the fixed coefficients of the Chebyshev polynomial seem to be near optimal and actually outperforming learned coefficients.

Polynomial degree Degree $d = 4$ achieves the lowest MASE out of degrees 2 through 8 that we tested. Interestingly, all of the degrees that we tested outperform the baseline model. See Figure 7.

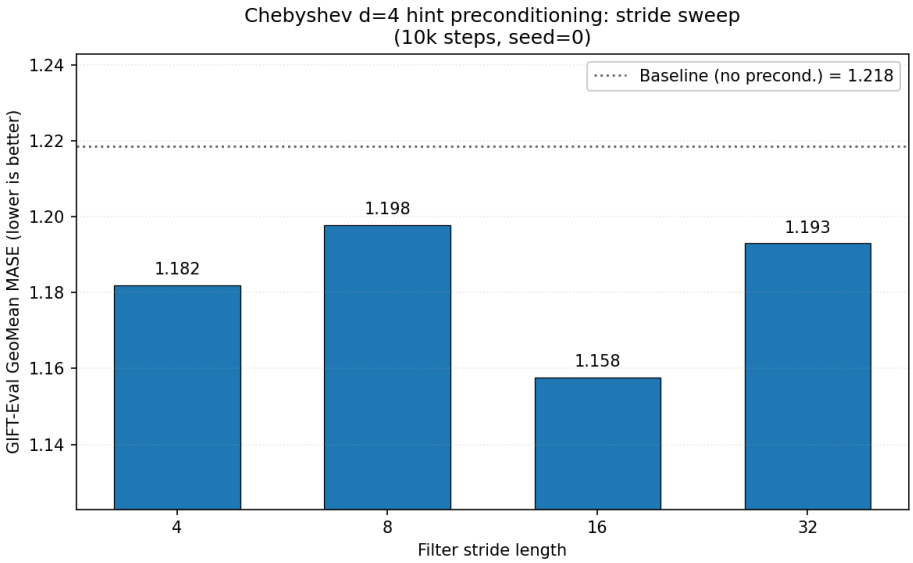

Figure 8: Filter stride ablation (seed 0, 97 configurations)

Filter stride As shown in Figure 8 we found that setting the stride to $s = 16$ is optimal. We hypothesize that this is because the stride is equal to the patch size of Moirai-2 Small, meaning that the convolutional filter is applied to corresponding time points in each patch.

Hint dropout We tested 5 other dropout probabilities and found 10% to be optimal. We tested each models performance when their hint channel was replaced with zeros, and found the expected result that increasing dropout causes less reliance on the hint. 10% dropout both reduced this performance degradation and lowered overall MASE.

5.4. Domain, Horizon, and Context Length

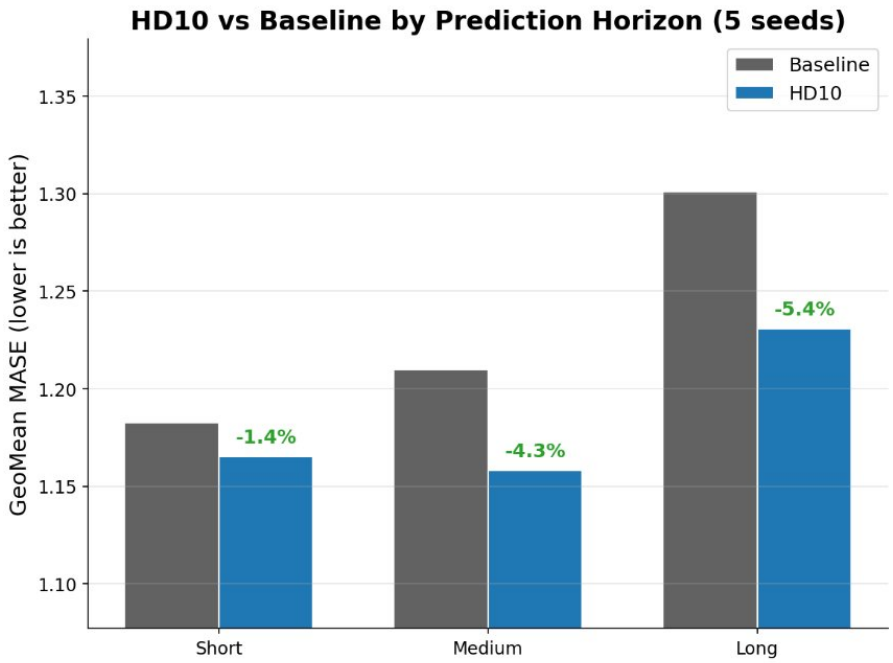

Figure 9: HD10 improvement over baseline broken down by the prediction horizon length. Horizon length is categorized by GIFT-eval, which groups each configuration into one of three prediction horizon lengths-short, medium, or long.

Figure 9 breaks down HD10's improvement by prediction horizon. We see that the relative improvement over the baseline model grows with increasing length of the prediction horizon. When grouped by series domain, we find that the healthcare, transport, and economics/finance domains have the largest performance improvements.

Figure 10 shows that the performance gain from HD10 stays similar across varying context lengths used in GIFT-eval, and that HD10 does not require a certain context length to be effective.

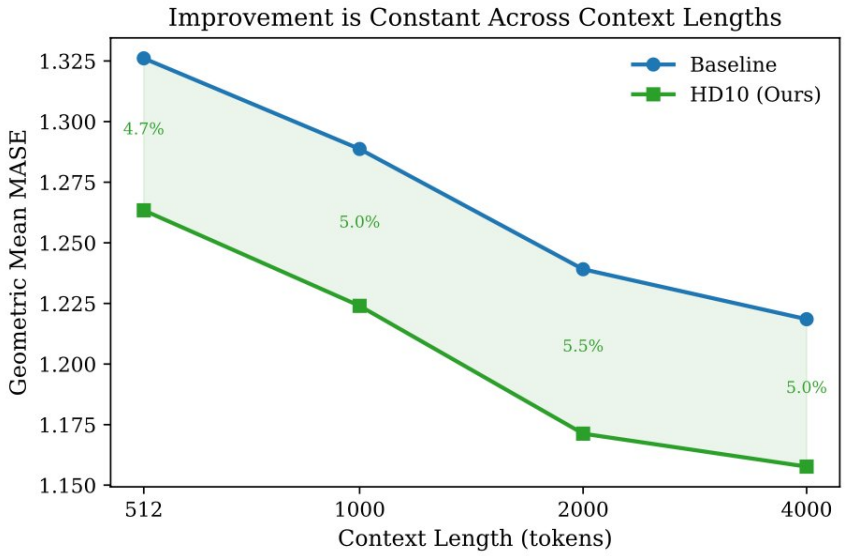

Figure 10: HD10 vs Baseline mean MASE using seed 0 as a function of context length. The relative improvement stays approximately constant across context lengths from 512 to 4,000 tokens.

6. Conclusions and Future Work

This paper tried to determine whether universal sequence preconditioning could improve zero shot time series forecasting models, and if so find a robust way to incorporate it. We found that the most direct analog to preconditioning in the online setting did not perform well because the effect of error accumulation that occurs when predicting over a longer horizon seemed to be stronger than the benefits seen by predicting in the preconditioned space. However, our *hint preconditioning* method was able to mitigate this problem. This idea is to apply the USP preconditioned signal to the model as an extra input channel, while keeping its predictions in the original space and loss calculated against the original target sequence. We found that this leads to a statistically significant 2.90% geometric mean MASE improvement on GIFT-Eval with a small increase in 0.11% to the model's parameter count.

We also perform ablations that confirm multiple choices we made in the design of hint preconditioning. For example, we confirmed the polynomial degree at $d = 4$, the optimal stride length set equal to the patch size, and a hint dropout that improves performance. Finally, we also tested learning the convolutional filter coefficients, and found that although this outperforms baseline, it

under performs the fixed coefficients of the chebyshev polynomial.

Limitations

- **Training data** We train all of our models on the publicly available LOTSA data corpus, which is only a subset of the data used to train the official Moirai-2 models. As mentioned, significant amounts of the official data is private to Salesforce and not available to us. As a result, our baseline model has a higher MASE than the official Moirai-2 Small model. This means that it is possible that the relative improvement of our hint preconditioning method may change when using this additional data and a larger pretraining scale.
- **Model scale** All main results use Moirai-2 Small (11.4M parameters). This was the best performing model in the Moirai-2 family on GIFT-eval. It is possible though that our results would change when implementing our technique with a larger model.
- **Model family** Our results are most directly applicable to the Moirai-2 models. Although our technique can be generally applied to zero-shot transformers models, we have not yet done this.
- **Benchmark** All results are on GIFT-Eval, which is a standard eval for zero-shot time series models. However, generalization to other time series benchmarks has not been tested.

Future work We propose a few directions that could either extend this work or address its current limitations. First, we could apply hint preconditioning to other foundation models including TimesFM and Chronos, two other high performing zero shot models, to see whether the benefit of our method can generalize across architectures. Second, it would be interesting to see whether the filter coefficients could be improved by learning domain-specific filters. The theory supports this idea because knowing the periodicity of the data, and therefore having information about the complex eigenvalues in the LDS analog could allow you to construct filters that better minimize these eigenvalues. Finally, the hint-channel framework may be applicable beyond time series, for example in language modeling where similar transformations could be performed on the token embedding inputs.

References

[1] T. Aksu *et al.*, "GIFT-Eval: A benchmark for general time series forecasting model evaluation," *arXiv preprint arXiv:2410.10393*, 2024.

[2] A. F. Ansari *et al.*, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024.

[3] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. Holden-Day, 1970.

[4] A. Das *et al.*, "A decoder-only foundation model for time-series forecasting," in *International Conference on Machine Learning (ICML)*, 2024.

[5] C. Liu *et al.*, "Moirai 2.0: When less is more for time series forecasting," *arXiv preprint arXiv:2511.11698*, 2026.

[6] A. Marsden and E. Hazan, "Universal sequence preconditioning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[7] G. Woo *et al.*, "Unified training of universal time series forecasting transformers," in *International Conference on Machine Learning (ICML)*, 2024.